

因之一是大多数测试人员开始误解“测试”一词的定义，他们认为：

- (1) 软件测试的目标是软件能够正确地完成预期的功能。
- (2) 软件测试就是证明软件不存在错误的过程。
- (3) 软件测试就是建立“软件做了其应该做的”信息的过程。

这些想法都是本末倒置的。软件测试应当是寻找软件的缺陷及漏洞，通过测试提高程序的价值、可靠性或质量。因此软件测试不应只是正向思维，也应该要逆向思维，甚至要发散思维。在测试开始时，测试人员应该假设程序中存在隐藏的错误，然后通过测试找到尽可能多的错误。软件测试就是通过不断地发现问题、解决问题来提高软件的质量。当然，软件测试的标准定义是软件测试使用手动或自动方式来运行或测试系统，其目的是测试其是否满足所需的要求或明确预期结果与实际结果之间的差异。

IEEE 对软件测试的定义是：使用人工或自动的手段来运行或测量软件系统的过程，目的是检验软件系统是否满足规定的需求，并找出与预期结果之间的差异。软件测试的发展趋势如下：

- (1) 测试工作将进一步前移。软件测试不仅是单元测试、集成测试、系统测试和验收测试，还有对需求的精确性和完整性的测试，以及对系统设计的测试。
- (2) 软件架构师、开发工程师、质量保证人员、测试工程师的工作将更好地融合。
- (3) 测试职业将得到更充分的尊重。
- (4) 设置独立的软件测试部门将成为未来软件公司的共识。
- (5) 测试外包服务将快速增长，和软件开发外包一样，软件测试外包将呈现全球化的趋势。

14.2.2 软件测试的基本问题

软件测试的基本问题是验证功能的实现和可用性，检查程序的错误，最终目的是提高用户体验，以较少的人力、物力和时间找出软件中潜在的各种错误和缺陷，通过修正各种错误和缺陷提高软件质量，回避软件发布后由于潜在的软件缺陷和错误造成的隐患所带来的商业风险。

同时，测试是以评价一个程序或者系统属性为目标的活动。测试是对软件质量的度量与评估，以验证软件的质量满足用户需求的程度，为用户选择和接收软件提供有力依据。

14.2.3 软件测试的对象及目的

软件测试的对象包括：源程序 / 目标代码；各开发阶段的文档（需求规格说明、概要设计说明、详细设计说明及其他相关文档）配置数据。

软件测试的目的包括：

- (1) 从用户角度看的目的：通过软件测试发现隐藏的错误和缺陷，考虑是否可以接受该产品。
- (2) 从开发者角度看的目的：表明软件产品不存在错误，验证软件实现了所有用户的要求。
- (3) 从测试人员角度看的目的：发现错误，预测错误，提供软件可靠性错误，对软件做出评价。

14.2.4 软件测试的原则

软件测试的原则包括以下几个方面：

(1) 测试用例的一个必要部分是预期输出和结果的定义。用例必须包括程序输入数据的两部分描述和输入数据下程序正确输出的准确描述。

(2) 程序员应该避免测试他们自己的程序。因为程序员很难以一种“破坏性”的视角来审查自己的程序，就如文档作者在编辑或校对自己的文档时一般无法找出错误的道理是一样的，程序员也存在同样的问题。

(3) 编写软件的组织不应当测试自己编写的软件。这类似于先前的一个论点，即软件项目或编程组织是一个与单个程序员有相似心理的有机组织，因此需要由更客观和独立的第三方进行测试。

(4) 应当具有足够的耐心及细心检查每一个测试结果。

(5) 测试用例不仅应基于有效和预期的输入，而且还应基于通常不会执行的无效和意外输入。当程序以新的或意想不到的方式运行时，软件产品中突然暴露出来的许多问题都会被发现。因此，有必要对不可预见的和无效的输入情况进行测试。

(6) 应避免重新设计测试用例。当软件需要重新测试时，重新设计用例需要投入大量的工作，所以测试用例的复用性及保留测试用例是有必要的。

(7) 计划测试工作不应默认假定不会发现错误，这是使用了不正确的软件测试定义的一个迹象。

(8) 测试不要穷举。穷举是指测试时考虑所有可能的输入值，若在测试过程中穷举所有数据组合需要耗费大量的时间。测试人员需要考虑程序逻辑的全覆盖，并确保程序中使用的所有条件都是可能的，并且以最少的用例达到最大的覆盖率。

(9) 软件测试要尽早执行。软件生命周期包括软件开发、测试、维护的整个流程，主要包括需求阶段、设计阶段、编码阶段、单元测试阶段、验收阶段和维修阶段。实践证明，在大多数情况下，需求分析阶段会出现缺陷，但维修成本很低，因此软件测试应尽快进行并贯穿整个软件生命周期。

(10) 软件测试应追溯到需求。软件测试的目标是发现问题。当然，满足用户的需求更为重要。

14.3 软件测试方法

按被测程序是否可见，可将软件测试方法分为黑盒测试（Black-box Testing）、白盒测试（White-box Testing）和灰盒测试（Gray-box Testing）；按是否需要执行被测程序，可将软件测试方法分为静态测试（Static Testing）和动态测试（Dynamic Testing）。

14.3.1 按照被测程序是否可见分类

1. 黑盒测试

黑盒测试又叫功能测试、数据驱动测试或基于需求规格说明书的功能测试。该类测试注重于测试软件的功能性需求。在测试中，把程序看作一个不能打开的黑盒子，在完全不考虑程序

内部结构和内部特性的情况下，在程序接口进行测试，它只检查程序功能是否按照需求规格说明书的规定正常使用，程序是否能适当地接收输入数据而产生正确的输出信息。黑盒测试着眼于程序外部结构，不考虑内部逻辑结构，主要针对软件界面和软件功能进行测试。

黑盒测试是从用户的角度，从输入数据与输出数据的对应关系出发进行测试的。很明显，如果外部特性本身设计有问题或规格说明的规定有误，用黑盒测试方法是发现不了的。

采用这种测试方法，测试工程师把测试对象看作一个黑盒子，完全不考虑程序内部的逻辑结构和内部特性，只依据程序的需求规格说明书检查程序的功能是否符合它的功能说明。测试工程师无须了解程序代码的内部构造，完全模拟软件产品的最终用户使用该软件，检查软件产品是否达到了用户的需求。黑盒测试方法能更好、更真实地从用户角度来考察被测系统的功能性需求实现情况。在软件测试的各个阶段，如单元测试、集成测试、系统测试及验收测试等阶段中，黑盒测试都发挥着重要作用，尤其在系统测试和确认测试中，其作用是其他测试方法无法取代的。

黑盒测试方法着重测试软件的功能需求，是在程序接口上进行的测试，主要是为了发现以下错误：

- (1) 是否有功能错误，是否有功能遗漏。
- (2) 是否能够正确地接收输入数据并产生正确的输出结果。
- (3) 是否有数据结构错误或外部信息访问错误。
- (4) 是否有程序初始化和终止方面的错误。

从理论上讲，黑盒测试只有采用穷举输入测试，把所有可能的输入都作为测试情况来考虑，才能查出程序中所有的错误。实际上测试情况有无穷多个，人们不仅要测试所有合法的输入，而且还要对那些不合法但可能的输入进行测试。这样看来，完全测试是不可能的，所以我们要进行有针对性的测试，通过制定测试案例指导测试的实施，保证软件测试有组织、按步骤、有计划地进行。黑盒测试行为必须能够加以量化，才能真正保证软件质量，而测试用例就是将测试行为具体量化的方法之一。具体的黑盒测试用例设计方法包括等价类划分法、边界值分析法、错误推测法、因果图法、判定表驱动法、正交试验设计法、功能图法和场景法等。

2. 白盒测试

白盒测试又称为结构测试、透明盒测试、逻辑驱动测试或基于代码的测试。白盒测试是一种测试用例设计方法，“盒子”指的是被测试的软件，“白盒”指的是盒子是可视的，即清楚盒子内部的东西以及里面是如何运作的。白盒测试法全面了解程序内部逻辑结构、对所有逻辑路径进行测试。白盒测试法是穷举路径测试。在使用这一方法时，测试者必须检查程序的内部结构，从检查程序的逻辑着手，得出测试数据。贯穿程序的独立路径数是天文数字。

白盒测试的测试方法有代码检查法、静态结构分析法、静态质量度量法、逻辑覆盖法、基本路径测试法、域测试、符号测试、路径覆盖和程序变异。

白盒测试法的覆盖标准有逻辑覆盖、循环覆盖和基本路径测试。其中，逻辑覆盖包括语句覆盖、判定覆盖、条件覆盖、判定/条件覆盖、条件组合覆盖和修改条件判断覆盖。这6种覆

盖标准发现错误的能力呈由弱到强变化。

- (1) 语句覆盖：每条语句至少执行一次。
- (2) 判定覆盖：每个判定的每个分支至少执行一次。
- (3) 条件覆盖：每个判定的每个条件应取到各种可能的值。
- (4) 判定 / 条件覆盖：同时满足判定覆盖条件覆盖。
- (5) 条件组合覆盖：每个判定中各条件的每一种组合至少出现一次。

(6) 修改条件判断覆盖：每一个判断的所有可能结果都出现过、每一个判断中所有条件的所有可能结果都出现过、每一个进入点及结束点都执行过、判断中每一个条件都可以独立影响判断的结果。

白盒测试的目的是通过检查软件内部的逻辑结构，对软件中的逻辑路径进行覆盖测试。在程序不同地方设立检查点，检查程序的状态，以确定实际运行状态与预期状态是否一致。

白盒测试需要遵循的原则如下：

- (1) 一个模块中的所有独立路径至少被测试一次。
- (2) 所有逻辑值均需测试 true 和 false 两种情况。
- (3) 检查程序的内部数据结构，保证其结构的有效性。
- (4) 在取值的上、下边界及可操作范围内运行所有循环。

白盒测试“三步法”具体如下：

- 第1步，根据代码的功能，人工设计测试用例进行基本功能测试。
- 第2步，统计白盒覆盖率，为未覆盖的白盒单位设计测试用例，实现完整的白盒覆盖，比较理想的覆盖率是实现100%的语句、条件、分支、路径覆盖。
- 第3步，自动生成大量的测试用例，捕捉“程序员未处理某些特殊输入”形成的错误。

第1步的测试用例通常是比较容易获得的，因为文档会规定程序的基本功能，没有文档的，程序员在编程时也要想清楚程序的功能，这些基本功能就是基本测试用例。

第2步是在第1步的基础上，检查未覆盖的白盒单位，由于未覆盖的逻辑单位通常对应未测试的等价类，因此第2步可以找出第1步所遗漏的测试用例。

第3步用自动动态测试弥补第2步的固有缺陷。

“三步法”尽量避免重复工作，将白盒测试方法和黑盒测试方法相结合，人工测试方法和自动测试方法相补充，如果第2步的覆盖率比较理想，那么基本上可以保证找出所有等价类。在开发过程允许的限度内，“三步法”已接近测试极限。

白盒测试主要用在具有高可靠性要求的软件领域，例如，军工软件、航天航空软件、工业控制软件等。在选购白盒测试工具时应当主要考虑对开发语言的支持、代码覆盖的深度、嵌入式软件的测试和测试的可视化等。

3. 灰盒测试

灰盒测试是介于白盒测试与黑盒测试之间的一种测试，灰盒测试多用于集成测试阶段，不仅关注输出、输入的正确性，同时也关注程序内部的情况。灰盒测试不像白盒那样详细、完整，但又比黑盒测试更关注程序的内部逻辑，常常是通过一些表征性的现象、事件、标志来判断内